2a. The two GenAI programs I compared were ChatGPT (OpenAI) and Gemini (Google). I did not use Copilot.

2b. I accessed ChatGPT through its website (chatgpt.com) in a web browser, and Gemini through its website (gemini.google.com) in a web browser. In each case, I started a new chat and pasted the prompt below as the first message.

2c. Write a single-file C++ program (C++17, compiles with g++ on Linux, file name Assignment4.cpp) that solves Sudoku puzzles using an object-oriented design (classes, not free functions) and recursive depth-first search with backtracking.
Requirements:

- A puzzle is a 9x9 grid in a text file: digits 1-9 are filled cells and an underscore (_) is a blank cell. Cells may or may not be separated by spaces.
- Each digit 1-9 must appear exactly once in every row, column, and 3x3 block.
- For each empty cell, compute the candidate digits; if there are none, backtrack; otherwise try each candidate recursively until the grid is full.
- If a puzzle has more than one solution, print ALL of them.
- Process puzzle1.txt through puzzle5.txt.
- For each file, print: the file name, the puzzle as read, and then each solution, or "No solution found" if there is none.
- Include comments explaining the code.

2d. ChatGPT

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <cctype>

using namespace std;

class Sudoku {
private:
    // A Sudoku board contains 9 rows and 9 columns.
    int board[9][9];

    // Stores the number of solutions found.
    int solutionCount;

public:
    // Constructor: initialize the board with zeros.
    Sudoku() {
        solutionCount = 0;
```

```
    for (int row = 0; row < 9; row++) {
        for (int col = 0; col < 9; col++) {
            board[row][col] = 0;
        }
    }
}

// Read the puzzle from a file.
// Digits represent filled cells, and '_' represents a blank.
// Whitespace is ignored, so spaces between cells are optional.
bool readPuzzle(const string& filename) {
    ifstream inputFile(filename);

    if (!inputFile.is_open()) {
        cerr << "Error: Could not open " << filename << endl;
        return false;
    }

    vector<char> cells;
    char ch;

    while (inputFile.get(ch)) {
        if (isspace(static_cast<unsigned char>(ch))) {
            continue;
        }

        // Only digits 1-9 and underscores are allowed.
        if ((ch >= '1' && ch <= '9') || ch == '_') {
            cells.push_back(ch);
        } else {
            cerr << "Error: Invalid character in "
                 << filename << endl;
            return false;
        }
    }

    // A Sudoku puzzle must contain exactly 81 cells.
    if (cells.size() != 81) {
        cerr << "Error: " << filename
             << " must contain exactly 81 cells." << endl;
        return false;
    }

    int index = 0;
```

```
    for (int row = 0; row < 9; row++) {
        for (int col = 0; col < 9; col++) {
            char cell = cells[index++];

            if (cell == '_') {
                board[row][col] = 0;
            } else {
                board[row][col] = cell - '0';
            }
        }
    }

    return true;
}

// Print the board.
// Zeros are displayed as underscores.
void printBoard() const {
    for (int row = 0; row < 9; row++) {
        for (int col = 0; col < 9; col++) {
            if (board[row][col] == 0) {
                cout << '_';
            } else {
                cout << board[row][col];
            }

            if (col == 2 || col == 5) {
                cout << " | ";
            } else if (col < 8) {
                cout << ' ';
            }
        }

        cout << endl;

        // Separate the 3x3 blocks horizontally.
        if (row == 2 || row == 5) {
            cout << "------+-------+------" << endl;
        }
    }
}

// Check whether a number already exists in the given row.
```

```
bool isInRow(int row, int number) const {
    for (int col = 0; col < 9; col++) {
        if (board[row][col] == number) {
            return true;
        }
    }

    return false;
}

// Check whether a number already exists in the given column.
bool isInColumn(int col, int number) const {
    for (int row = 0; row < 9; row++) {
        if (board[row][col] == number) {
            return true;
        }
    }

    return false;
}

// Check whether a number already exists in the 3x3 block.
bool isInBlock(int row, int col, int number) const {
    // Find the top-left corner of the 3x3 block.
    int startRow = (row / 3) * 3;
    int startCol = (col / 3) * 3;

    for (int r = startRow; r < startRow + 3; r++) {
        for (int c = startCol; c < startCol + 3; c++) {
            if (board[r][c] == number) {
                return true;
            }
        }
    }

    return false;
}

// A number is valid if it does not appear in the
// cell's row, column, or 3x3 block.
bool isValid(int row, int col, int number) const {
    return !isInRow(row, number)
        && !isInColumn(col, number)
        && !isInBlock(row, col, number);
```

```
}

// Verify that the original puzzle has no duplicate digits
// in any row, column, or 3x3 block.
bool isInitialBoardValid() const {
    for (int row = 0; row < 9; row++) {
        for (int col = 0; col < 9; col++) {
            int number = board[row][col];

            if (number == 0) {
                continue;
            }

            // Temporarily remove the digit so that the
            // validation methods can check its position.
            const_cast<Sudoku*>(this)->board[row][col] = 0;

            bool valid = isValid(row, col, number);

            const_cast<Sudoku*>(this)->board[row][col] = number;

            if (!valid) {
                return false;
            }
        }
    }

    return true;
}

// Compute all candidate digits for an empty cell.
vector<int> getCandidates(int row, int col) const {
    vector<int> candidates;

    for (int number = 1; number <= 9; number++) {
        if (isValid(row, col, number)) {
            candidates.push_back(number);
        }
    }

    return candidates;
}

// Recursive depth-first search with backtracking.
```

```
void solve() {
    // Find the next empty cell.
    int emptyRow = -1;
    int emptyCol = -1;

    for (int row = 0; row < 9; row++) {
        for (int col = 0; col < 9; col++) {
            if (board[row][col] == 0) {
                emptyRow = row;
                emptyCol = col;
                break;
            }
        }

        if (emptyRow != -1) {
            break;
        }
    }

    // No empty cells remain, so a complete solution was found.
    if (emptyRow == -1) {
        solutionCount++;

        cout << "\nSolution " << solutionCount << ":" << endl;
        printBoard();
        return;
    }

    // Find all valid candidates for the selected empty cell.
    vector<int> candidates = getCandidates(emptyRow, emptyCol);

    // No candidates means this path cannot produce a solution.
    if (candidates.empty()) {
        return;
    }

    // Try every candidate recursively.
    for (int number : candidates) {
        // Make a choice.
        board[emptyRow][emptyCol] = number;

        // Explore this choice.
        solve();
```

```
            // Undo the choice to explore other possibilities.
            board[emptyRow][emptyCol] = 0;
        }
    }

    // Solve the puzzle and report the result.
    void solvePuzzle() {
        solutionCount = 0;

        // Reject puzzles that already violate Sudoku rules.
        if (!isInitialBoardValid()) {
            cout << "No solution found" << endl;
            return;
        }

        solve();

        // If the search found no complete board, there is no solution.
        if (solutionCount == 0) {
            cout << "No solution found" << endl;
        } else {
            cout << "\nTotal solutions found: "
                 << solutionCount << endl;
        }
    }
};

int main() {
    // Process all five required puzzle files.
    for (int i = 1; i <= 5; i++) {
        string filename = "puzzle" + to_string(i) + ".txt";

        cout << "\n========================================" << endl;
        cout << "File: " << filename << endl;
        cout << "========================================" << endl;

        Sudoku sudoku;

        // Read and display the original puzzle.
        if (!sudoku.readPuzzle(filename)) {
            continue;
        }

        cout << "\nOriginal puzzle:" << endl;
```

```
        sudoku.printBoard();

        // Find and print all possible solutions.
        cout << "\nSolving " << filename << "..." << endl;
        sudoku.solvePuzzle();
    }

    return 0;
}
```

Gemini

```
/**
 * Assignment4.cpp
 *
 * An object-oriented Sudoku solver implemented in C++17.
 * Processes puzzle1.txt through puzzle5.txt, computes candidates,
 * uses recursive backtracking DFS, and prints all valid solutions.
 *
 * Compile with: g++ -std=c++17 Assignment4.cpp -o Assignment4
 * Run with:     ./Assignment4
 */

#include <iostream>
#include <fstream>
#include <vector>
#include <string>

class SudokuBoard {
private:
    // 9x9 grid where 0 represents an empty cell, and 1-9 represent filled cells.
    std::vector<std::vector<int>> grid;

    /**
     * Checks if placing a number at a specific row and column is valid
     * according to Sudoku rules (row, column, and 3x3 block uniqueness).
     */
    bool isValid(int row, int col, int num) const {
        // Check row and column constraints
        for (int i = 0; i < 9; ++i) {
            if (grid[row][i] == num) return false;
            if (grid[i][col] == num) return false;
        }
```

```
    // Check 3x3 subgrid constraints
    int startRow = row - (row % 3);
    int startCol = col - (col % 3);
    for (int r = 0; r < 3; ++r) {
        for (int c = 0; c < 3; ++c) {
            if (grid[startRow + r][startCol + c] == num) {
                return false;
            }
        }
    }

    return true;
}

/**
 * Computes all valid candidate digits (1-9) for a given empty cell.
 */
std::vector<int> getCandidates(int row, int col) const {
    std::vector<int> candidates;
    for (int num = 1; num <= 9; ++num) {
        if (isValid(row, col, num)) {
            candidates.push_back(num);
        }
    }
    return candidates;
}

/**
 * Recursive depth-first search helper with backtracking to find all solutions.
 */
void solveRecursive(std::vector<std::vector<std::vector<int>>>& solutions) {
    int emptyRow = -1;
    int emptyCol = -1;
    bool foundEmpty = false;

    // Find the next empty cell (value 0)
    for (int r = 0; r < 9 && !foundEmpty; ++r) {
        for (int c = 0; c < 9 && !foundEmpty; ++c) {
            if (grid[r][c] == 0) {
                emptyRow = r;
                emptyCol = c;
                foundEmpty = true;
            }
        }
```

```
        }

        // If no empty cells remain, a complete solution has been found
        if (!foundEmpty) {
            solutions.push_back(grid);
            return;
        }

        // Compute candidates for the current empty cell
        std::vector<int> candidates = getCandidates(emptyRow, emptyCol);

        // If there are no candidates, this path fails (backtrack)
        if (candidates.empty()) {
            return;
        }

        // Try each valid candidate recursively
        for (int num : candidates) {
            grid[emptyRow][emptyCol] = num;
            solveRecursive(solutions);
            grid[emptyRow][emptyCol] = 0; // Backtrack
        }
    }

public:
    // Constructor initializes an empty 9x9 board
    SudokuBoard() : grid(9, std::vector<int>(9, 0)) {}

    /**
     * Loads a puzzle from a text file, handling digits 1-9 and underscores (_) for blanks.
     * Whitespace and newlines are safely ignored.
     */
    bool loadFromFile(const std::string& filename) {
        std::ifstream infile(filename);
        if (!infile.is_open()) {
            return false;
        }

        grid.assign(9, std::vector<int>(9, 0));
        int r = 0, c = 0;
        char ch;

        while (infile >> std::noskipws >> ch && r < 9) {
            if (ch == '_' || (ch >= '1' && ch <= '9')) {
```

```
            grid[r][c] = (ch == '_') ? 0 : (ch - '0');
            c++;
            if (c == 9) {
                c = 0;
                r++;
            }
        }
    }
    return true;
}

/**
 * Prints a grid in a formatted 9x9 structure with 3x3 block separators.
 */
void printGrid(const std::vector<std::vector<int>>& targetGrid) const {
    for (int r = 0; r < 9; ++r) {
        for (int c = 0; c < 9; ++c) {
            if (targetGrid[r][c] == 0) {
                std::cout << "_ ";
            } else {
                std::cout << targetGrid[r][c] << " ";
            }
            if ((c + 1) % 3 == 0 && c < 8) {
                std::cout << "| ";
            }
        }
        std::cout << "\n";
        if ((r + 1) % 3 == 0 && r < 8) {
            std::cout << "------+-------+------\n";
        }
    }
}

// Prints the current board state
void printBoard() const {
    printGrid(grid);
}

/**
 * Initiates the solving process and returns a list of all found solutions.
 */
std::vector<std::vector<std::vector<int>>> solve() {
    std::vector<std::vector<std::vector<int>>> solutions;
    solveRecursive(solutions);
```

```
        return solutions;
    }
};

int main() {
    // Process puzzle1.txt through puzzle5.txt sequentially
    for (int i = 1; i <= 5; ++i) {
        std::string filename = "puzzle" + std::to_string(i) + ".txt";

        std::cout << "========================================\n";
        std::cout << "File Name: " << filename << "\n";
        std::cout << "========================================\n";

        SudokuBoard board;
        if (!board.loadFromFile(filename)) {
            std::cout << "Error: Could not open file " << filename << ".\n\n";
            continue;
        }

        std::cout << "Puzzle as read:\n";
        board.printBoard();
        std::cout << "\n";

        // Solve the puzzle and retrieve all solutions
        auto solutions = board.solve();

        if (solutions.empty()) {
            std::cout << "No solution found.\n\n";
        } else {
            std::cout << "Solutions found: " << solutions.size() << "\n";
            for (size_t s = 0; s < solutions.size(); ++s) {
                std::cout << "Solution " << (s + 1) << ":\n";
                board.printGrid(solutions[s]);
                std::cout << "\n";
            }
        }
    }

    return 0;
}
```

2e.

- Correctness: Both programs find every solution on valid puzzles. They produced the same results on the Wikipedia puzzle (one solution) and on the two-solution puzzle (both

solutions were printed). The difference is in how they handle bad input. When I gave each program a completed grid with a repeated 5 in one row, ChatGPT printed "No solution found," which is correct, because it checks the given digits before searching. Gemini reported the grid as one valid solution, which is wrong, because it never checks the given digits. When I gave each program a file with only 80 cells, ChatGPT rejected it with an error message, while Gemini accepted it and solved it as if the last cell were blank. By reading Gemini's loading loop, I also found that it silently skips any character other than 1-9 and underscore instead of reporting an error, whereas ChatGPT rejects the file. Finally, when no solution exists, ChatGPT prints "No solution found," matching the assignment text, while Gemini prints "No solution found." with an extra period. Overall, ChatGPT's code is more correct because it validates the input and the given digits, while Gemini's code can print an invalid grid as a "solution."

- Execution time: Both programs have the same worst-case running time, roughly O(9^k) for k empty cells, because they fill cells in the same order and re-scan 27 cells for every candidate check. Measured on my test machine (Linux, g++ -O2), both finished the Wikipedia example in under 0.01 seconds. On the hard puzzle, ChatGPT took about 7.7 seconds and Gemini about 8.0 seconds. That difference is small and within normal run-to-run variation, so neither is meaningfully faster. Both are far slower than necessary on hard puzzles, which is why improving the search order matters.

- Space complexity: ChatGPT's program prints each solution as soon as it is found, so its memory use stays constant: the board plus a recursion depth of at most 81 frames. It does allocate a small vector<int> of candidates at every recursive call. Gemini's program stores every solution in a vector<vector<vector<int>>>, so its memory grows as O(s) for s solutions, and each stored 9x9 board is ten separate heap allocations. A puzzle with a very large number of solutions could run out of memory. ChatGPT's approach is better here.

- Maintainability: ChatGPT's code is split into small single-purpose methods (isInRow, isInColumn, isInBlock) with clear comments, so it is easy to read and test. Its weaknesses are using namespace std, a const_cast inside isInitialBoardValid that gets around const, and helper methods that are public when they should be private. Gemini's code has a cleaner style, with std:: prefixes, private helpers, and a single isValid method. However, its nested-vector solution type is hard to read, loading and validation are mixed together, and errors are not reported, which makes bugs hard to find.

2f. I selected the ChatGPT program as the basis for my assignment. It is more correct because it validates the file contents and the given digits, which Gemini's program does not, and Gemini's program can print an invalid grid as a solution. It also uses constant memory for output instead of storing every solution. The two programs run at about the same speed, so that did not favor either one. ChatGPT's weaknesses (the const_cast, slow cell selection, and public helper methods) are small and easy to fix, while fixing Gemini's would have required adding validation and redesigning how solutions are stored.

2g. Correctness: I removed the const_cast hack. Duplicate given digits are now detected while the file is read by checking each given digit against the digits already used in its row, column, and block. The file reader now rejects files with more than 81 cells as well as fewer, and still rejects illegal characters. A puzzle whose given digits already break a rule is printed and reported as "No solution found" without searching. The program prints "No solution found" exactly as the assignment specifies and still prints every solution when a puzzle has several.

Execution time: I replaced the repeated row, column, and block scans with three arrays of bitmasks that record which digits are already used, so a candidate check is now a few bit operations instead of a scan of 27 cells. I also replaced "first empty cell" with "empty cell with the fewest candidates," so the search runs into dead ends much sooner and explores far fewer branches. On my test machine, the hard puzzle dropped from about 7.7 seconds with the original ChatGPT code to about 0.007 seconds, with identical solutions on all of my test puzzles.

Space complexity: Candidates are now held in a single int bitmask instead of a vector allocated at every recursive call, so the search does no heap allocation. Solutions are still printed the moment they are found and are not stored, so memory does not grow with the number of solutions.

Maintainability: I split the single class into SudokuBoard, which holds the grid and constraints, and SudokuSolver, which performs the recursive search, so each class has one job. I removed using namespace std, made the helper data private, and added place and remove methods so the board's bookkeeping lives in one place. I also added a prologue, block comments, and comments that explain each line, with each block labeled by where it came from.